04_models_rf_xgboost

Contents

1 04 Random Forest and XGBoost **1**
1.1 1. Libraries . . . 1
1.2 2. Constants . . . 1
1.3 3. Load Data and Reproduce Split . . . 2
1.4 4. Metrics Helper . . . 3
1.5 5. Random Forest . . . 4
1.6 6. XGBoost . . . 10
1.7 7. Results Summary . . . 16
1.8 8. Save Outputs . . . 16

1 04 Random Forest and XGBoost

Author: Aaron **Purpose:** Estimate Random Forest (RF) and XGBoost models across all three prediction horizons (1-day, 3-day, 5-day). Hyperparameter tuning via Latin hypercube grid search over rolling-origin cross-validation folds. Outputs feature importance plots, model objects, and a performance metrics table.

Depends on: `data/processed/features.csv` (output of `02_feature_engineering.Rmd`)

1.1 1. Libraries

```r
library(tidymodels)    # parsnip, recipes, tune, rsample
library(ranger)        # RF engine
library(xgboost)       # gradient boosting engine
library(vip)           # variable importance plots
library(kableExtra)    # LaTeX table export
library(tidyverse)
library(here)
```

1.2 2. Constants

All configuration values are defined once here and referenced throughout. No magic numbers inline.

```r
# Reproducibility
RANDOM_SEED <- 42L

# Data split - must mirror 02_feature_engineering.Rmd Section 10
TRAIN_PROP  <- 0.80

# Rolling-origin CV
CV_INITIAL  <- 200L   # minimum training window; covers MACD init ~35 obs + buffer
CV_ASSESS   <- 60L    # assessment window (~one trading quarter)
```

```r
CV_SKIP     <- 100L   # gives ~13 slices on 80% of 1850 obs; increase to speed up tuning

# Random Forest
RF_TREES     <- 500L
RF_GRID_SIZE <- 15L   # Latin hypercube grid points
RF_MTRY_MIN  <- 3L
RF_MTRY_MAX  <- 13L   # floor(27 predictors / 2), rule of thumb for regression RF
RF_MINN_MIN  <- 5L
RF_MINN_MAX  <- 30L

# XGBoost
XGB_TREES     <- 500L
XGB_GRID_SIZE <- 20L  # Latin hypercube grid points for 5 parameters

# Column definitions
TARGET_COLS  <- c("target_1d", "target_3d", "target_5d")
HORIZON_LABS <- c("1d",        "3d",        "5d")

PREDICTOR_COLS <- c(
  # contemporaneous returns (known at time t)
  "r_bpi", "r_bdo", "r_psei", "r_fx", "r_us10y",
  # trend
  "ma5", "ma10", "ma20", "ema12", "ema26", "ma_ratio",
  # momentum
  "lag1", "lag2", "lag3", "mom5", "mom10", "rsi14", "macd", "macd_signal",
  # volatility
  "vol5", "vol10", "vol20", "vol_ratio",
  # cross-asset
  "lag_psei",
  # engineered transformations
  "interact", "vol5_sq",
  # regime
  "hi_vol"
)
```

1.3 3. Load Data and Reproduce Split

The 80/20 chronological split replicates the exact partition defined in `02_feature_engineering.Rmd` (Section 10). All five model scripts must use the identical cutoff row to ensure comparable out-of-sample periods.

```r
features <- read_csv(
  here("data", "processed", "features.csv"),
  show_col_types = FALSE
)

n_total <- nrow(features)
n_train <- floor(TRAIN_PROP * n_total)
n_test  <- n_total - n_train

train_df <- features[seq_len(n_train), ]
test_df  <- features[(n_train + 1L):n_total, ]
```

```r
cat("Total clean observations:", n_total, "\n")
```

```
## Total clean observations: 2623
```

```r
cat("Training set            :", n_train,
    "| up to", format(train_df$date[nrow(train_df)], "%Y-%m-%d"), "\n")
```

```
## Training set            : 2098 | up to 2024-11-01
```

```r
cat("Test set                :", n_test,
    "| from", format(test_df$date[1], "%Y-%m-%d"), "\n")
```

```
## Test set                : 525 | from 2024-11-02
```

1.4 4. Metrics Helper

Computes RMSE, MAE, R^2 (out-of-sample), Directional Accuracy, and Naïve Directional Accuracy for one model-horizon evaluation. The naïve benchmark is the previous day's BPI log return (`lag1`) as a stand-in prediction.

```r
compute_metrics <- function(test_actual, test_pred, train_actual, train_pred, naive_pred, model, horizo
  test_ss_res   <- sum((test_actual - test_pred)^2)
  test_ss_total <- sum((test_actual - mean(test_actual))^2)
  train_ss_res  <- sum((train_actual - train_pred)^2)
  train_ss_total <- sum((train_actual - mean(train_actual))^2)

  tibble(
    Model      = model,
    Horizon    = horizon,
    RMSE       = sqrt(mean((test_actual - test_pred)^2)),
    MAE        = mean(abs(test_actual - test_pred)),
    R2         = 1 - test_ss_res / test_ss_total,
    DA         = mean(sign(test_pred) == sign(test_actual)),
    Naive_DA   = mean(sign(naive_pred) == sign(test_actual)),
    Train_RMSE = sqrt(mean((train_actual - train_pred)^2)),
    Train_MAE  = mean(abs(train_actual - train_pred)),
    Train_R2   = 1 - train_ss_res / train_ss_total,
    Train_DA   = mean(sign(train_pred) == sign(train_actual))
  )
}

# QVRS Economist Theme
theme_quant <- function() {
  theme_minimal() +
    theme(
      text = element_text(color = "#0D0D0D"),
      plot.title = element_text(face = "bold", size = 14, margin = margin(b = 5)),
      plot.subtitle = element_text(size = 11, color = "#333333", margin = margin(b = 10)),
      plot.caption = element_text(size = 8, color = "#595959", hjust = 0, margin = margin(t = 10)),
      axis.title = element_text(size = 10, face = "bold"),
      axis.text = element_text(size = 9),
      panel.grid.minor = element_blank(),
      panel.grid.major = element_line(color = "#F2F2F2", linewidth = 0.3),
      legend.position = "bottom",
```

```
      axis.ticks = element_blank()
    )
}
```

1.5 5. Random Forest

Tree-based ensembles are scale-invariant; no feature normalisation is applied. We tune two hyperparameters:

- **`mtry`** (features sampled per split): controls decorrelation between trees.
- **`min_n`** (minimum leaf size): regularises tree depth.

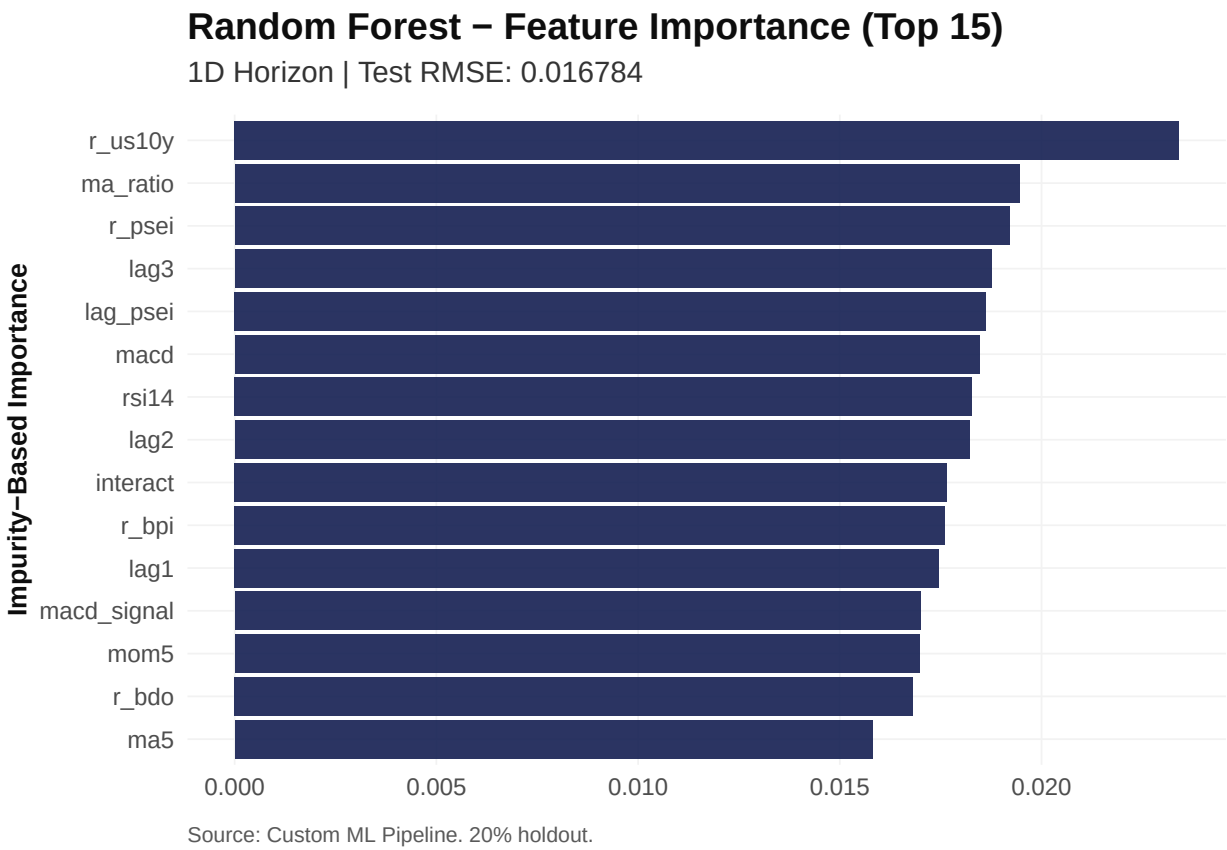
All 500 trees use impurity-based importance scoring (`importance = "impurity"`) for feature importance extraction via `vip`.

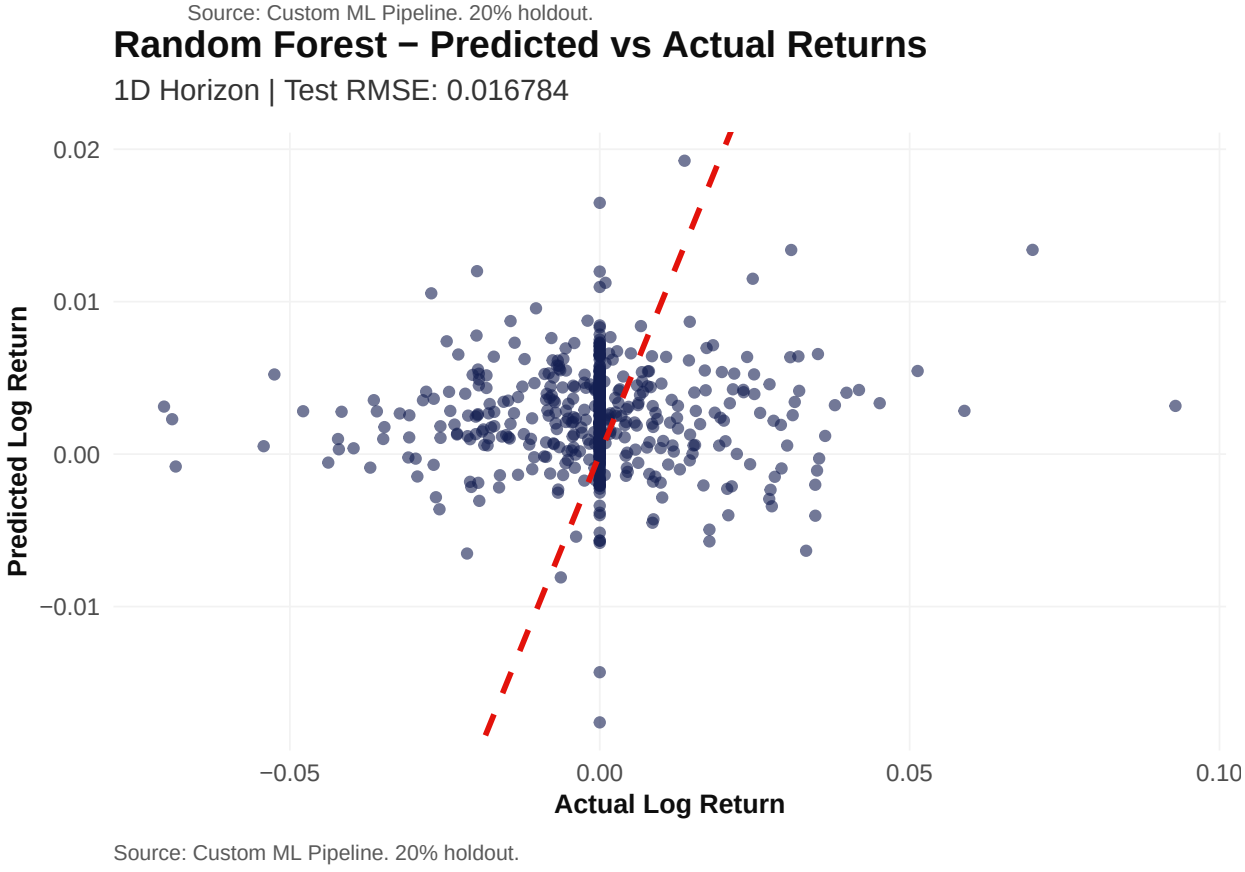
```
set.seed(RANDOM_SEED)

rf_metrics_list <- vector("list", length(TARGET_COLS))

for (h in seq_along(TARGET_COLS)) {

  target_col    <- TARGET_COLS[h]
  horizon_label <- HORIZON_LABS[h]

  cat("\nRF |", toupper(horizon_label), "horizon\n")

  # Select only the outcome + predictors to keep other target columns out of the recipe
  train_h <- train_df %>% select(all_of(c(target_col, PREDICTOR_COLS)))
  test_h  <- test_df  %>% select(all_of(c(target_col, PREDICTOR_COLS)))

  # Rolling-origin CV on training set only - no test data ever enters the fold
  cv_h <- rolling_origin(
    data       = train_h,
    initial    = CV_INITIAL,
    assess     = CV_ASSESS,
    skip       = CV_SKIP,
    cumulative = TRUE
  )
  cat("CV slices:", nrow(cv_h), "\n")

  # No normalisation needed - trees are scale-invariant
  rf_recipe <- recipe(
    as.formula(paste(target_col, "~ .")),
    data = train_h
  )

  rf_spec <- rand_forest(
    mtry  = tune(),
    min_n = tune(),
    trees = RF_TREES
  ) %>%
    set_engine(
      "ranger",
      importance = "impurity",   # required for vip() to extract importance scores
```

```r
    seed       = RANDOM_SEED
  ) %>%
  set_mode("regression")

rf_wf <- workflow() %>%
  add_recipe(rf_recipe) %>%
  add_model(rf_spec)

rf_grid <- grid_latin_hypercube(
  mtry(range  = c(RF_MTRY_MIN, RF_MTRY_MAX)),
  min_n(range = c(RF_MINN_MIN, RF_MINN_MAX)),
  size = RF_GRID_SIZE
)

rf_tune <- tune_grid(
  rf_wf,
  resamples = cv_h,
  grid      = rf_grid,
  metrics   = metric_set(rmse, mae, rsq),
  control   = control_grid(verbose = TRUE, save_pred = FALSE)
)

best_rf <- select_best(rf_tune, metric = "rmse")
cat("Best mtry:", best_rf$mtry, "| Best min_n:", best_rf$min_n, "\n")

final_rf_wf  <- finalize_workflow(rf_wf, best_rf)
final_rf_fit <- fit(final_rf_wf, data = train_h)

rf_preds       <- predict(final_rf_fit, new_data = test_h)$.pred
rf_train_preds <- predict(final_rf_fit, new_data = train_h)$.pred
naive_preds    <- test_df$lag1   # previous day's return as naive benchmark

rf_metrics_list[[h]] <- compute_metrics(
  test_actual  = test_h[[target_col]],
  test_pred    = rf_preds,
  train_actual = train_h[[target_col]],
  train_pred   = rf_train_preds,
  naive_pred   = naive_preds,
  model        = "Random Forest",
  horizon      = horizon_label
)
cat("RMSE:", round(rf_metrics_list[[h]]$RMSE, 6),
    "| DA:",  round(rf_metrics_list[[h]]$DA,   4),
    "| Naive DA:", round(rf_metrics_list[[h]]$Naive_DA, 4), "\n")

# 1. Feature Importance Plot
vip_plot <- final_rf_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 15, aesthetics = list(fill = "#141F52", alpha = 0.9)) +
  labs(
    title    = "Random Forest - Feature Importance (Top 15)",
    subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(rf_metrics_list[[h]]$RMS
    caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
```

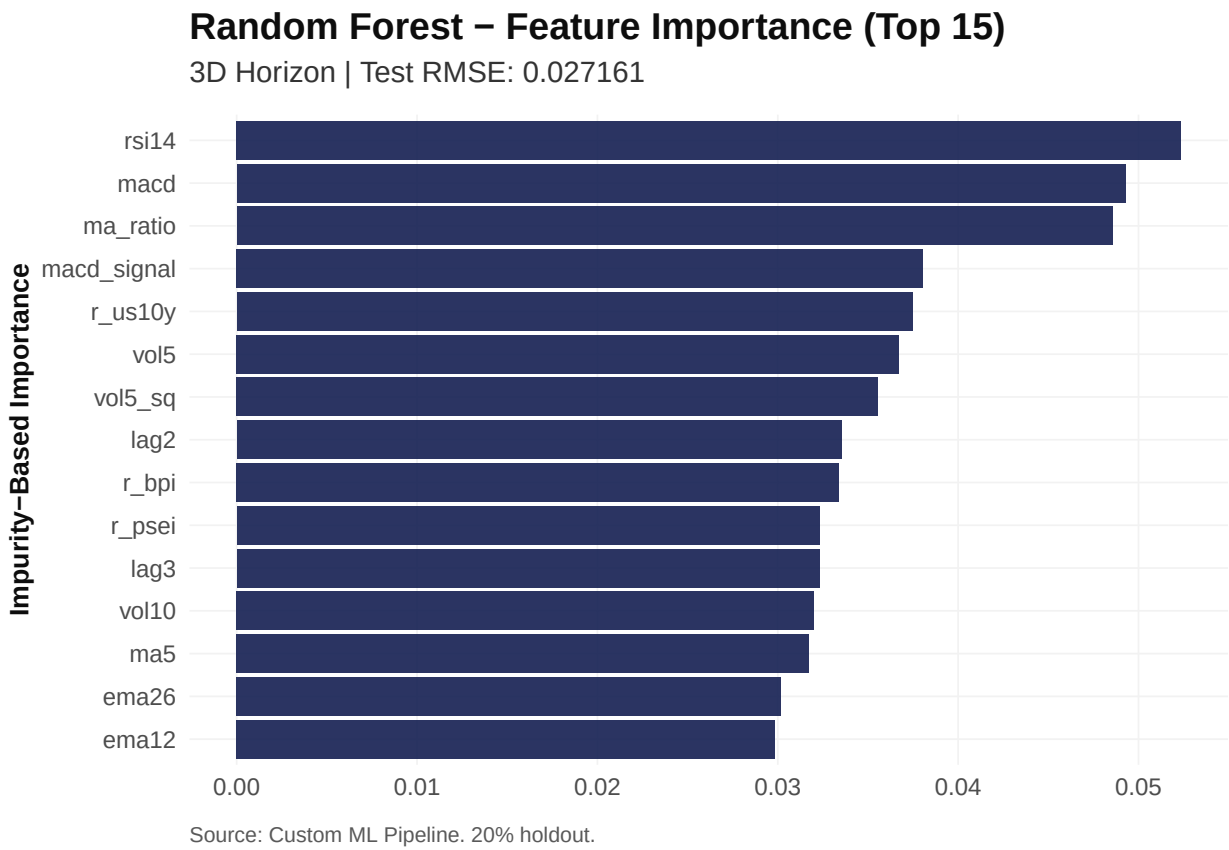
```r
      x = "Impurity-Based Importance",
      y = NULL
    ) +
    theme_quant()

  ggsave(
    here("paper", "figures", paste0("rf_importance_", horizon_label, ".png")),
    vip_plot,
    width = 8, height = 6, dpi = 300
  )

  # 2. Predicted vs Actual Plot
  pred_vs_actual_plot <- ggplot(tibble(Actual = test_h[[target_col]], Predicted = rf_preds), aes(x = Act
    geom_point(color = "#141F52", alpha = 0.6) +
    geom_abline(intercept = 0, slope = 1, color = "#E3120B", linetype = "dashed", linewidth = 1) +
    labs(
      title    = "Random Forest - Predicted vs Actual Returns",
      subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(rf_metrics_list[[h]]$RMS
      caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
      x = "Actual Log Return",
      y = "Predicted Log Return"
    ) +
    theme_quant()

  ggsave(
    here("paper", "figures", paste0("rf_pred_actual_", horizon_label, ".png")),
    pred_vs_actual_plot,
    width = 8, height = 6, dpi = 300
  )

  saveRDS(
    final_rf_fit,
    here("output", "models", paste0("rf_", horizon_label, ".rds")),
  )
  cat("Outputs saved for RF", horizon_label, "\n")
}

rf_metrics <- bind_rows(rf_metrics_list)
```

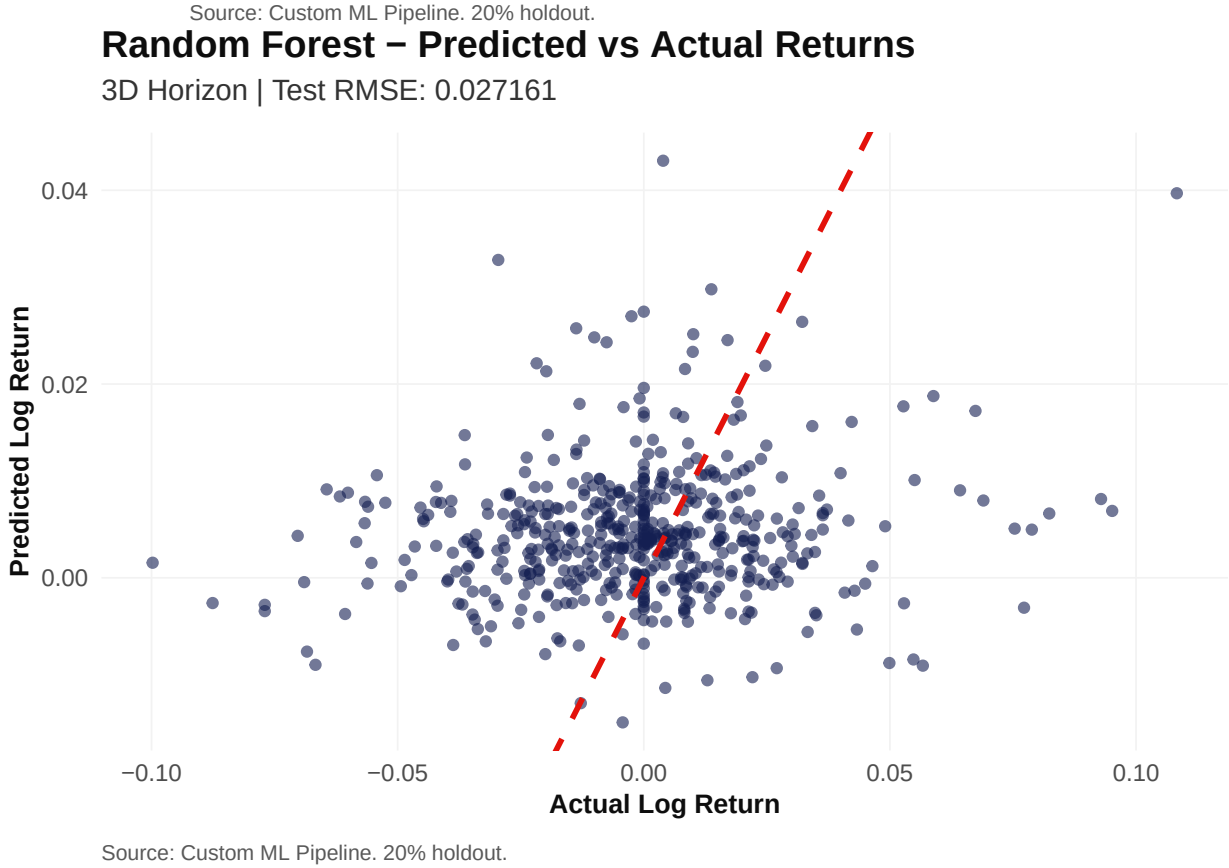
```
## RF 1D | RMSE: 0.016784 | DA: 0.2895
```



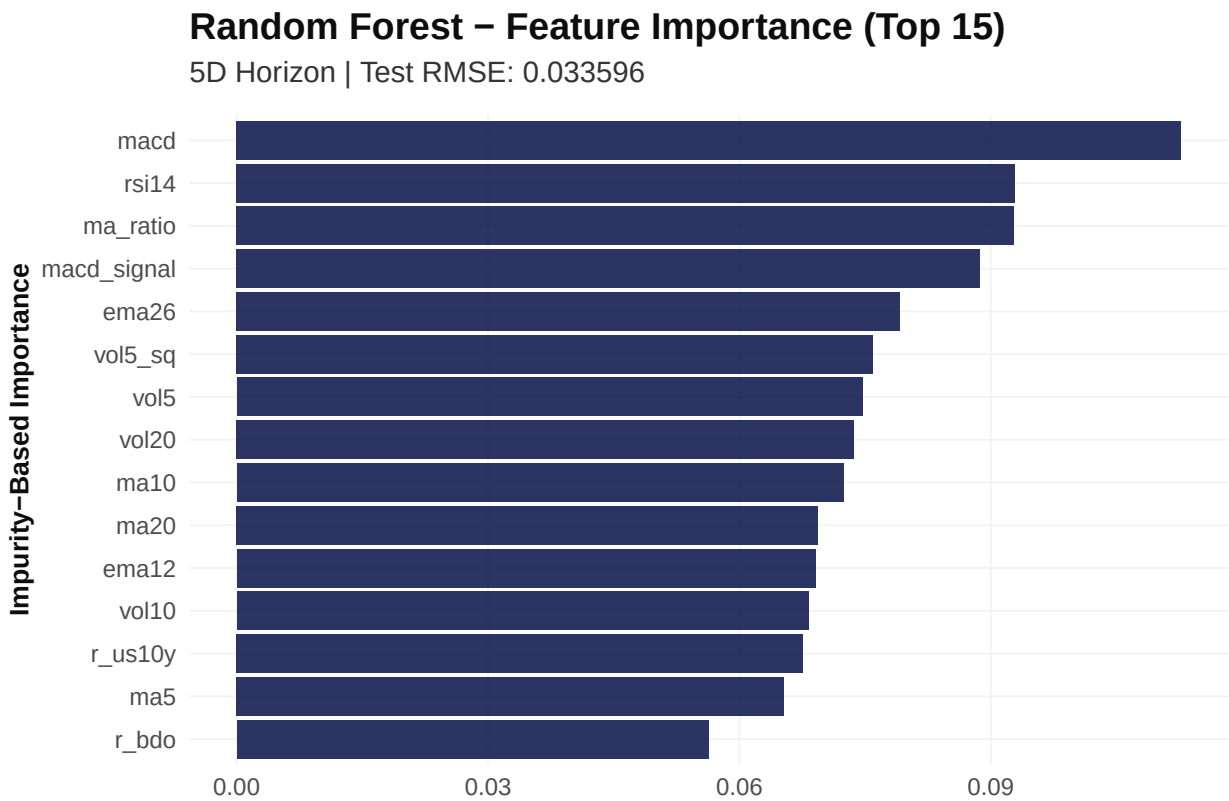



```
## RF 3D | RMSE: 0.027161 | DA: 0.4667
```

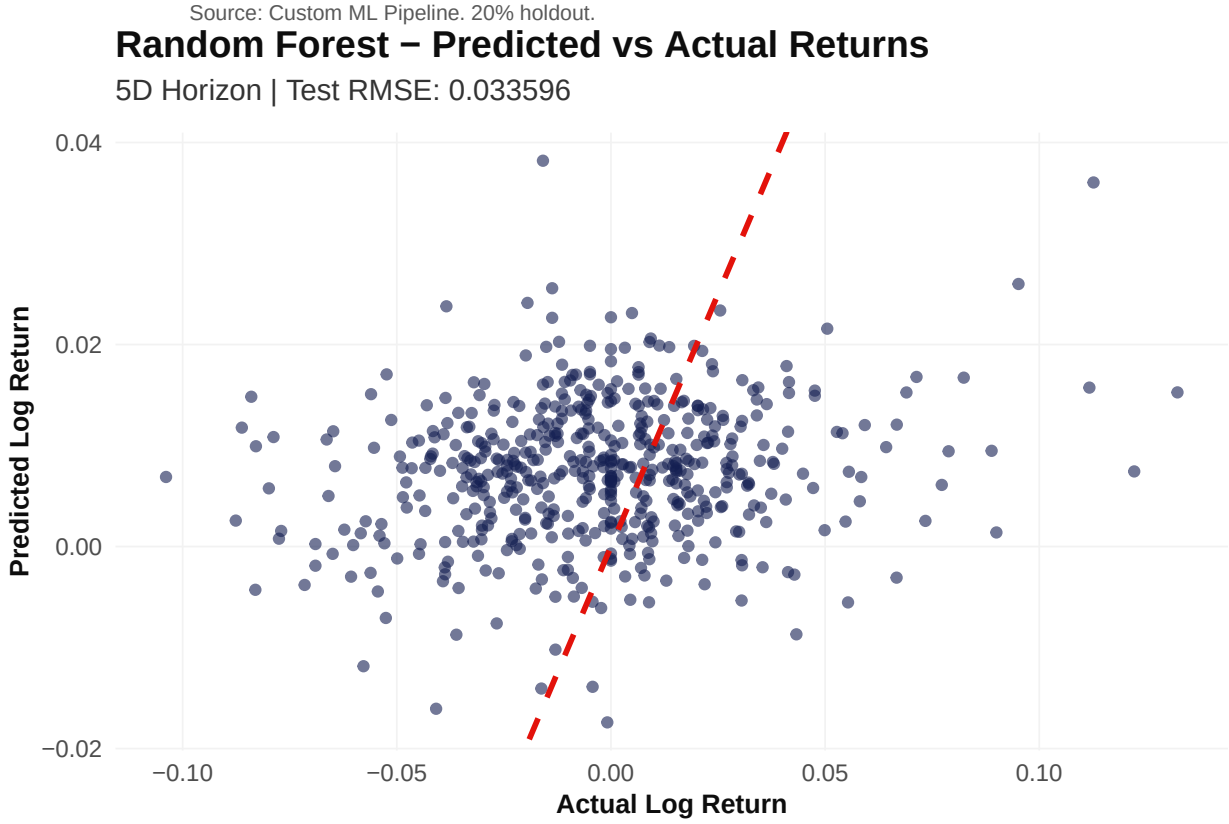





```
## RF 5D | RMSE: 0.033596 | DA: 0.4838
```



Source: Custom ML Pipeline. 20% holdout.



Source: Custom ML Pipeline. 20% holdout.

1.6 6. XGBoost

XGBoost is the primary tuned model. We tune five hyperparameters via a 20-point Latin hypercube grid:

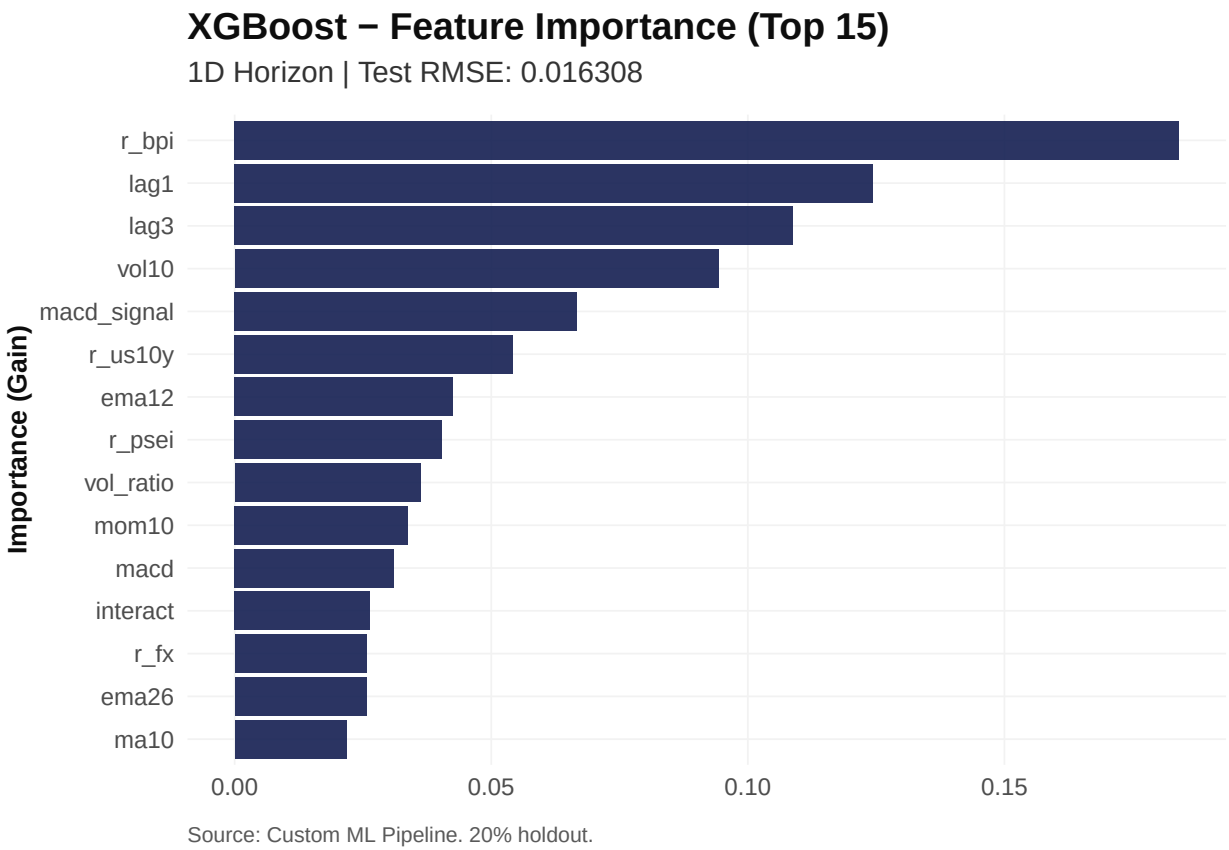
Parameter	Controls
`learn_rate`	Step size shrinkage; prevents overfitting
`tree_depth`	Complexity of individual trees
`min_n`	Minimum observations in a node
`loss_reduction`	Gamma; minimum loss reduction to split
`stop_iter`	Early stopping rounds

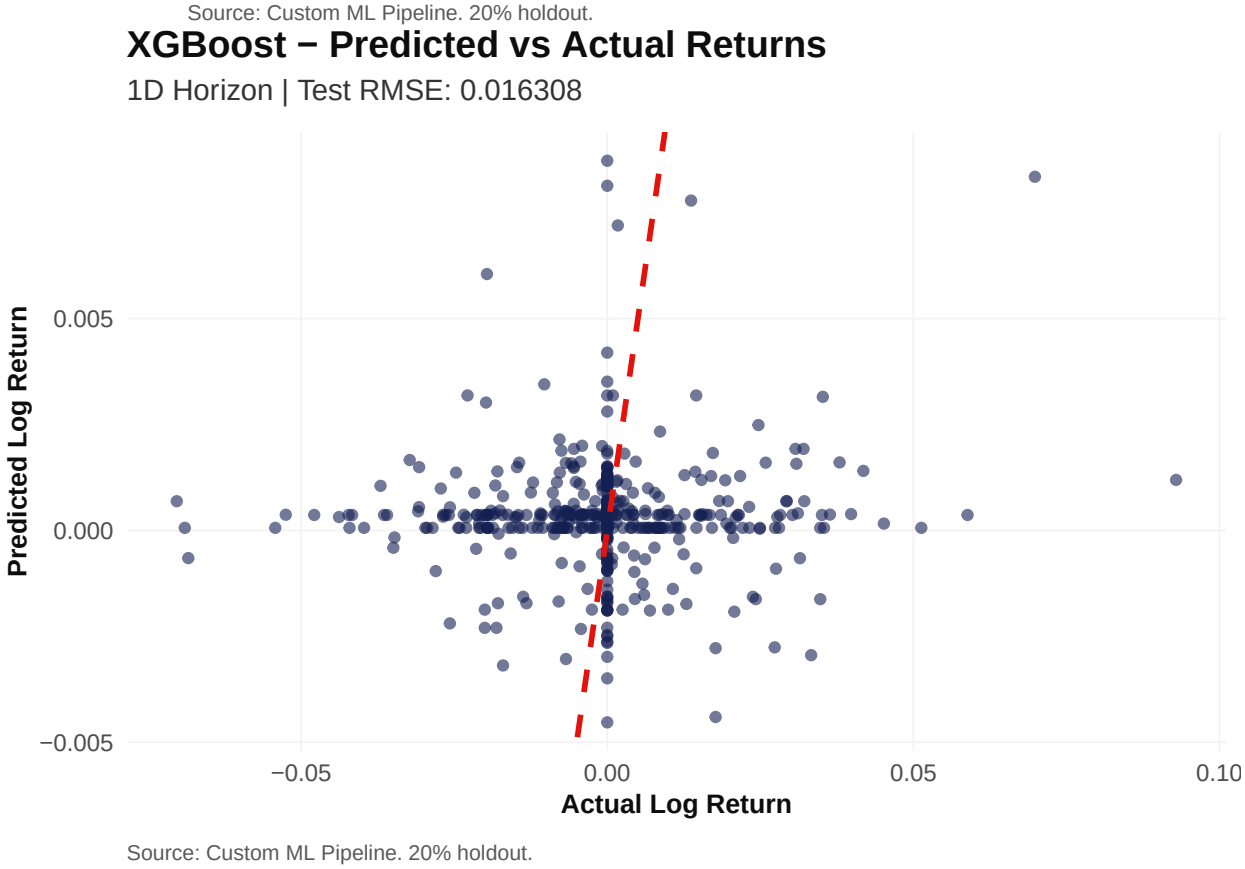
```r
set.seed(RANDOM_SEED)

xgb_metrics_list <- vector("list", length(TARGET_COLS))

for (h in seq_along(TARGET_COLS)) {

  target_col    <- TARGET_COLS[h]
  horizon_label <- HORIZON_LABS[h]

  cat("\nXGBoost |", toupper(horizon_label), "horizon\n")

  train_h <- train_df %>% select(all_of(c(target_col, PREDICTOR_COLS)))
  test_h  <- test_df  %>% select(all_of(c(target_col, PREDICTOR_COLS)))

  cv_h <- rolling_origin(
    data       = train_h,
    initial    = CV_INITIAL,
    assess     = CV_ASSESS,
    skip       = CV_SKIP,
    cumulative = TRUE
  )
  cat("CV slices:", nrow(cv_h), "\n")

  xgb_recipe <- recipe(
    as.formula(paste(target_col, "~ .")),
    data = train_h
  )

  xgb_spec <- boost_tree(
    trees          = XGB_TREES,
    learn_rate     = tune(),
    tree_depth     = tune(),
    min_n          = tune(),
    loss_reduction = tune(),
    stop_iter      = tune()
  ) %>%
    set_engine("xgboost", nthread = parallel::detectCores()) %>%
    set_mode("regression")

  xgb_wf <- workflow() %>%
    add_recipe(xgb_recipe) %>%
    add_model(xgb_spec)
```

```r
xgb_grid <- grid_latin_hypercube(
  learn_rate(range = c(-3, -1)), # 0.001 to 0.1
  tree_depth(range = c(3L, 8L)),
  min_n(range      = c(5L, 30L)),
  loss_reduction(),
  stop_iter(range  = c(10L, 50L)),
  size = XGB_GRID_SIZE
)

xgb_tune <- tune_grid(
  xgb_wf,
  resamples = cv_h,
  grid      = xgb_grid,
  metrics   = metric_set(rmse, mae, rsq),
  control   = control_grid(verbose = TRUE, save_pred = FALSE)
)

best_xgb <- select_best(xgb_tune, metric = "rmse")
cat("Best learn_rate:", round(best_xgb$learn_rate, 5),
    "| tree_depth:", best_xgb$tree_depth,
    "| min_n:", best_xgb$min_n, "\n")

final_xgb_wf  <- finalize_workflow(xgb_wf, best_xgb)
final_xgb_fit <- fit(final_xgb_wf, data = train_h)

xgb_preds       <- predict(final_xgb_fit, new_data = test_h)$.pred
xgb_train_preds <- predict(final_xgb_fit, new_data = train_h)$.pred
naive_preds     <- test_df$lag1

xgb_metrics_list[[h]] <- compute_metrics(
  test_actual  = test_h[[target_col]],
  test_pred    = xgb_preds,
  train_actual = train_h[[target_col]],
  train_pred   = xgb_train_preds,
  naive_pred   = naive_preds,
  model        = "XGBoost",
  horizon      = horizon_label
)
cat("RMSE:", round(xgb_metrics_list[[h]]$RMSE, 6),
    "| DA:",  round(xgb_metrics_list[[h]]$DA,   4),
    "| Naive DA:", round(xgb_metrics_list[[h]]$Naive_DA, 4), "\n")

# 1. Feature Importance Plot
vip_plot <- final_xgb_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 15, aesthetics = list(fill = "#141F52", alpha = 0.9)) +
  labs(
    title    = "XGBoost - Feature Importance (Top 15)",
    subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(xgb_metrics_list[[h]]$RM
    caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
    x = "Importance (Gain)",
    y = NULL
  ) +
```

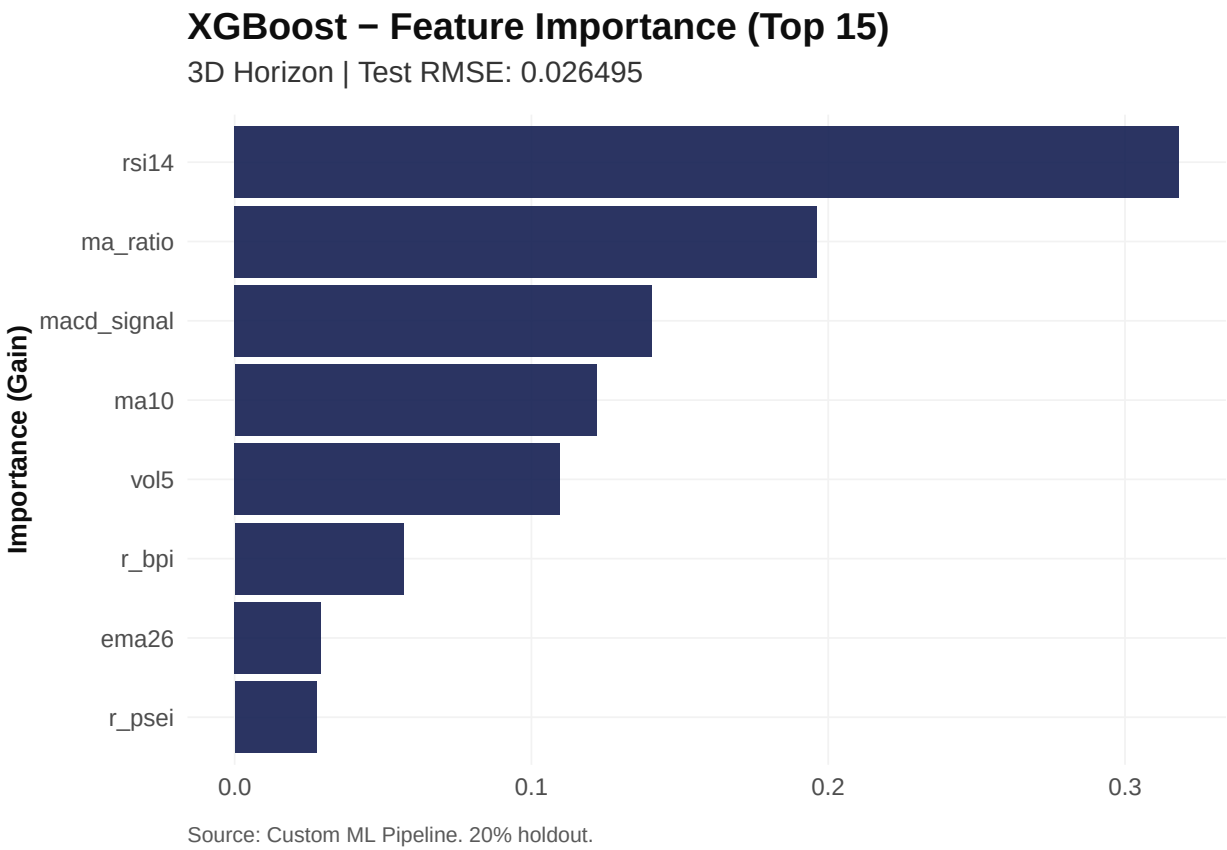
```r
    theme_quant()

  ggsave(
    here("paper", "figures", paste0("xgboost_importance_", horizon_label, ".png")),
    vip_plot,
    width = 8, height = 6, dpi = 300
  )

  # 2. Predicted vs Actual Plot
  pred_vs_actual_plot <- ggplot(tibble(Actual = test_h[[target_col]], Predicted = xgb_preds), aes(x = A
    geom_point(color = "#141F52", alpha = 0.6) +
    geom_abline(intercept = 0, slope = 1, color = "#E3120B", linetype = "dashed", linewidth = 1) +
    labs(
      title    = "XGBoost - Predicted vs Actual Returns",
      subtitle = paste0(toupper(horizon_label), " Horizon | Test RMSE: ", round(xgb_metrics_list[[h]]$RM
      caption  = "Source: Custom Machine Learning Pipeline. Out-of-sample testing on 20% holdout.",
      x = "Actual Log Return",
      y = "Predicted Log Return"
    ) +
    theme_quant()

  ggsave(
    here("paper", "figures", paste0("xgboost_pred_actual_", horizon_label, ".png")),
    pred_vs_actual_plot,
    width = 8, height = 6, dpi = 300
  )

  saveRDS(
    final_xgb_fit,
    here("output", "models", paste0("xgboost_", horizon_label, ".rds")),
  )
  cat("Outputs saved for XGBoost", horizon_label, "\n")
}

xgb_metrics <- bind_rows(xgb_metrics_list)
```

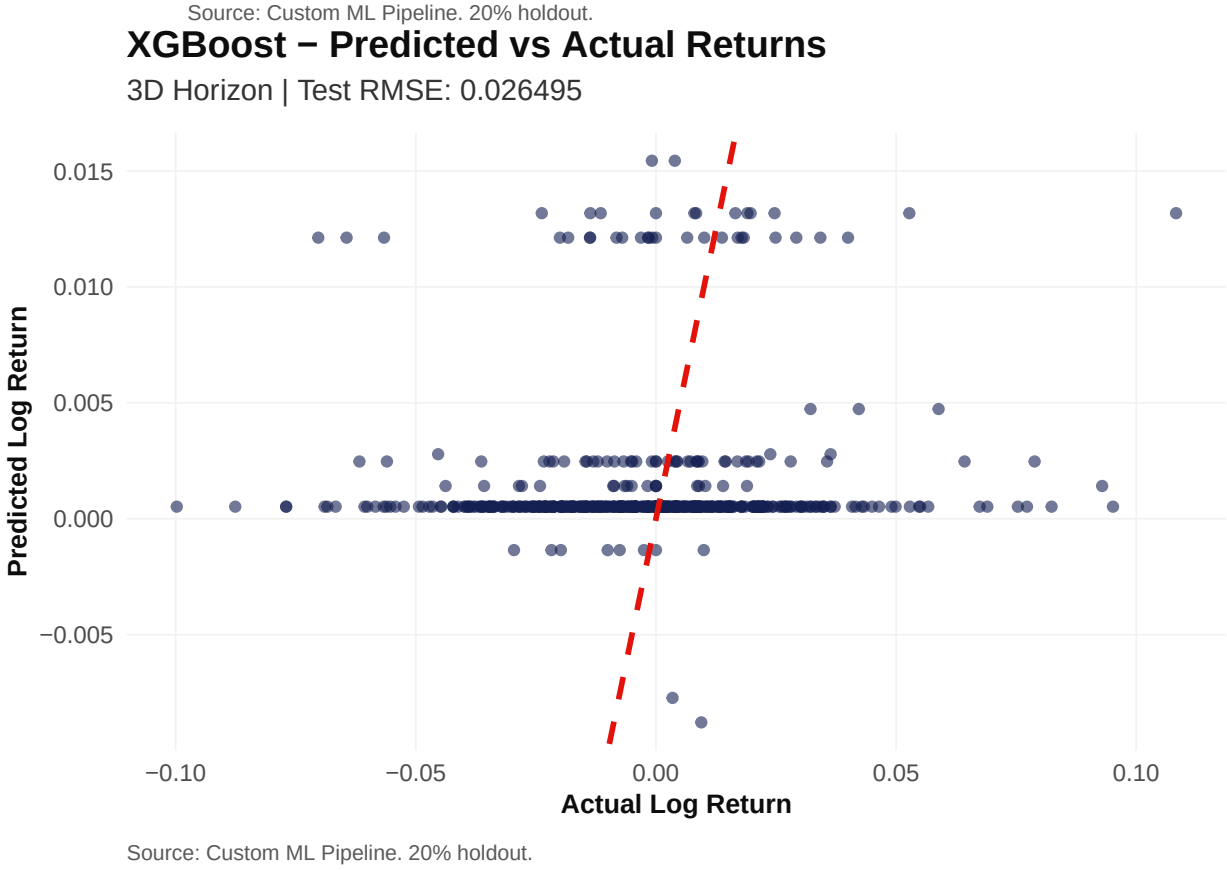
```
## XGBoost 1D | RMSE: 0.016308 | DA: 0.2781
```



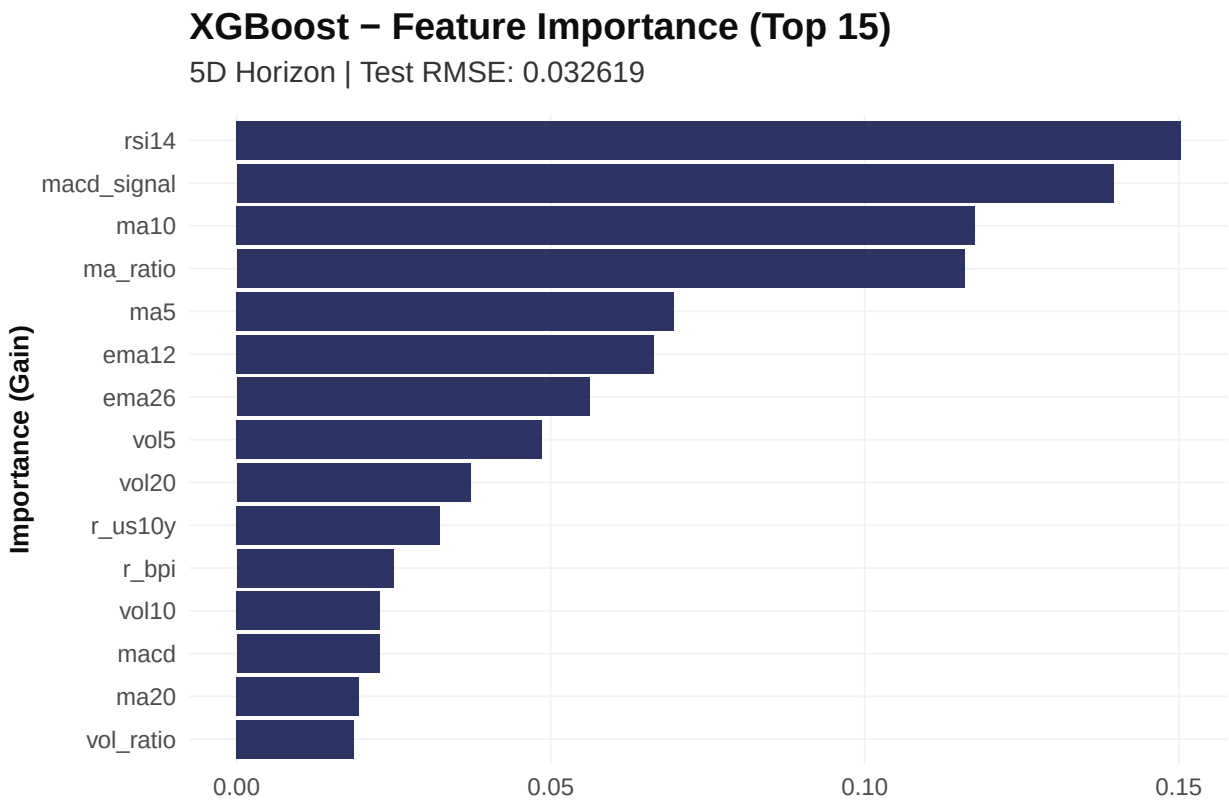



```
## XGBoost 3D | RMSE: 0.026495 | DA: 0.4533
```

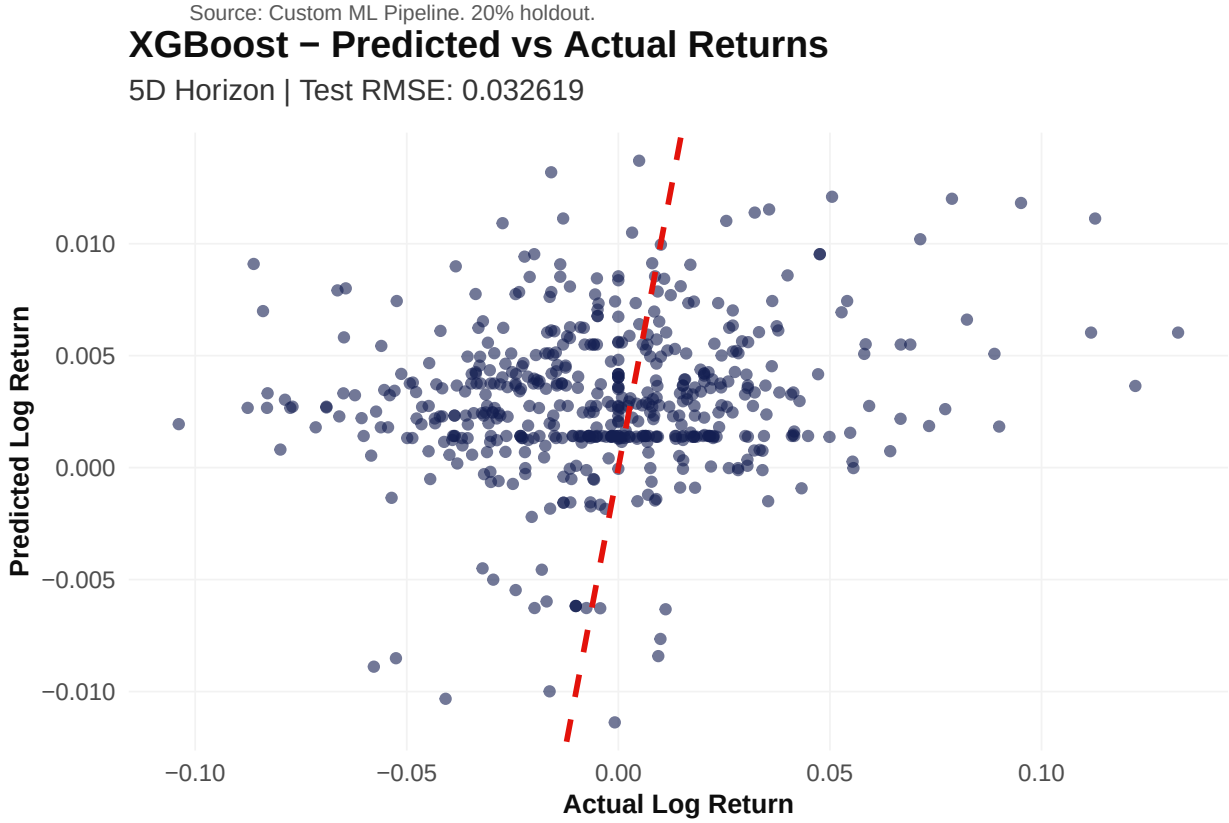





```
## XGBoost 5D | RMSE: 0.032619 | DA: 0.4819
```



Source: Custom ML Pipeline. 20% holdout.



Source: Custom ML Pipeline. 20% holdout.

1.7 7. Results Summary

```
all_metrics <- bind_rows(rf_metrics, xgb_metrics) %>%
  mutate(across(where(is.double), ~ round(., 5)))

kable(
  all_metrics,
  caption   = "Random Forest and XGBoost: Out-of-Sample Performance by Horizon",
  col.names = c("Model", "Horizon", "RMSE", "MAE", "R2", "DA", "Naive DA", "Train RMSE", "Train MAE", "T
  digits    = 5
) %>%
  kable_styling(bootstrap_options = c("striped", "condensed"), full_width = FALSE)
```

Table 1: Random Forest and XGBoost: Out-of-Sample Performance by Horizon

Model	Horizon	RMSE	MAE	R2	DA	Naive DA	Train RMSE	Train MAE	Train R2
Random Forest	1d	0.01678	0.01120	-0.06034	0.28952	0.27619	0.01454	0.00959	0.20721
Random Forest	3d	0.02716	0.02037	-0.06425	0.46667	0.32000	0.02298	0.01694	0.22610
Random Forest	5d	0.03360	0.02613	-0.07917	0.48381	0.30286	0.02757	0.02070	0.30144
XGBoost	1d	0.01631	0.00998	-0.00099	0.27810	0.27619	0.01610	0.00964	0.02776
XGBoost	3d	0.02649	0.01923	-0.01265	0.45333	0.32000	0.02533	0.01811	0.05925
XGBoost	5d	0.03262	0.02470	-0.01734	0.48190	0.30286	0.03049	0.02218	0.14607

1.8 8. Save Outputs

```
write_csv(all_metrics, here("output", "predictions", "rf_xgboost_metrics.csv"))
message("Metrics saved to output/predictions/rf_xgboost_metrics.csv")

if (!dir.exists(here("paper", "tables"))) dir.create(here("paper", "tables"))

latex_tbl <- kable(
  all_metrics,
  format    = "latex",
  booktabs  = TRUE,
  escape    = TRUE,
  linesep   = "",
  col.names = c("Model", "Horizon", "RMSE", "MAE", "R2", "DA", "Naive DA", "Train RMSE", "Train MAE", "T
  digits    = 5
)

# Build threeparttable manually: footnote outside table sizing
rf_xgb_note <- paste0(
  "DA = Directional Accuracy (fraction of test-set observations where the ",
  "predicted sign matches the realised sign). ",
  "Naive DA uses the previous day's return (lag1) as the prediction. ",
  "R2 is out-of-sample and may be negative if the model performs worse than the mean."
)

latex_tbl <- kable(
```

```r
  all_metrics,
  format    = "latex",
  booktabs  = TRUE,
  escape    = TRUE,
  linesep   = "",
  col.names = c("Model", "Horizon", "RMSE", "MAE", "R2", "DA", "Naive DA", "Train RMSE", "Train MAE", "T
  digits    = 5
)

# Build table manually: footnote outside table sizing
rf_xgb_note <- paste0(
  "DA = Directional Accuracy (fraction of test-set observations where the ",
  "predicted sign matches the realised sign). ",
  "Naive DA uses the previous day's return (lag1) as the prediction. ",
  "R2 is out-of-sample and may be negative if the model performs worse than the mean."
)

latex_tbl <- paste0(
  "\\begin{table}[!ht]\n\\centering\n",
  "\\caption{\\label{tab:rf_xgboost_metrics}Random Forest and XGBoost: Out-of-Sample Performance by Hori
  "\\footnotesize\n",
  "\\setlength{\\tabcolsep}{3.5pt}\n",
  as.character(latex_tbl), "\n",
  "\\multicolumn{11}{p{\\linewidth}}{\\rule{0pt}{1em}\\textit{Note.} ", rf_xgb_note, "}\\\\\\\n",
  "\\end{tabular}\n",
  "\\end{table}"
)

writeLines(latex_tbl, here("paper", "tables", "rf_xgboost_metrics.tex"))
message("LaTeX table saved to paper/tables/rf_xgboost_metrics.tex")
message("04_models_rf_xgboost.Rmd: Complete.")
```